Session 2.1

The Learning Problem: a Handwritten Digit, and the Neuron

What it means to learn a function from examples, and the smallest unit that does it



Redistribution prohibited without permission
@Zhigang Feng

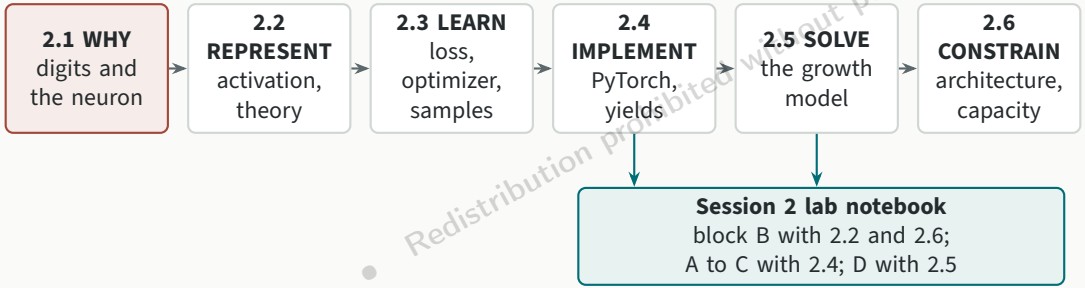
Zhigang Feng

What You Will Be Able to Do After Session 2

- **Read a network as a function**: write $f_\theta(x)$, count its parameters, and say what its architecture assumes about the input.
- **Check one training step by hand against autograd**: forward, loss, backward, update (Deck 2.4).
- **Write the PyTorch training loop** and check it on a problem whose answer you already know.
- **Run an honest forecast comparison**: splits by date, one information set, a random-walk baseline.
- **Solve a growth model with a network**, and show with held-out tests that the solution is right.
- **Throughout: a small training loss is a claim about the training sample, and nothing more.**

© Zhigang Feng · Redistribution prohibited without permission

Session 2 in One Map



© Thibaut Feng
Redistribution prohibited without permission

- The running example for 2.1 to 2.3 is one picture: a handwritten digit.
- The economic applications are in 2.4 (a forecast) and 2.5 (a model solved).
- **One habit in all six decks: test a method first where the answer is known.**

What Session 1 Already Gave Us, and What It Only Named

- **Four verbs** for what AI does in economics: measure, predict, identify, compute (Session 1.2). Session 2 is about *predict* and *compute*.
- **Three kinds of learning**: supervised (inputs with labels), unsupervised (inputs only), reinforcement (learning by acting). Session 2 is supervised; reinforcement is Session 3.
- **Empirical risk minimisation**: choose θ to minimise the average loss $\frac{1}{n}\sum_i \ell\big(f_\theta(x_i), y_i\big)$ over a sample (Session 1.3). OLS is the linear, squared-loss case.
- Session 1 only *named* the rule that makes this honest: judge a model on data it never saw. Deck 2.3 turns it into a procedure, and every result after it is reported that way.
- **Today starts one step earlier: with a problem no formula solves.**

© Zhitang Feng · Redistribution prohibited without permission

Session 2.1

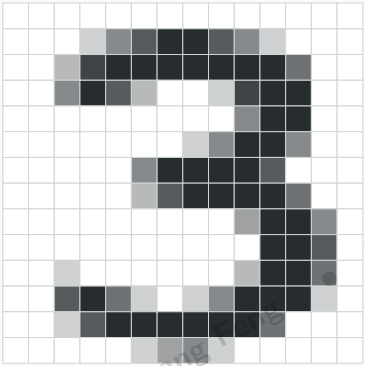
A Handwritten Digit

One picture carries the ideas of Decks 2.1 to 2.3



© Zhigang Feng Redistribution prohibited without permission

A Handwritten Digit

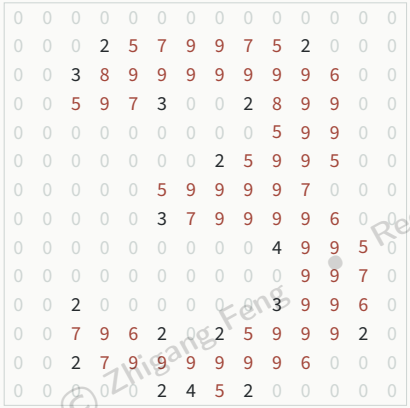


- You read this as a “3” at once, and you cannot say how.
- The benchmark is **MNIST**: 70,000 digits written by US Census Bureau staff and high-school students, each a 28×28 grey-scale image with its label.
- Drawn here at 14×14, so that every pixel can be seen.
- **Nobody writes that function down. It is learned from labelled examples.**

© Zhigang Feng
Redistribution prohibited without permission

Source: Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, “Gradient-based learning applied to document recognition”, *Proc. IEEE* 86(11), 1998.

The Same Thing, Written Down



- To a computer the image is a table of ink levels: 0 is blank paper, 9 is full ink (0 to 255 in the original files).
- Read row after row, the table is a vector: 196 numbers here, $28 \times 28 = 784$ in MNIST. That vector is the entire input.
- Flattening forgets which pixels are neighbours. Deck 2.6 puts that back.
- **An image is a point in a space with one dimension per pixel: 196 here, 784 in MNIST.**

© Zhigang Feng Redistribution prohibited without permission

The Learning Problem: from 784 Numbers to Ten Classes



© Thiago Pérez — Redistribution prohibited without permission

- **Data:** 60,000 labelled pairs (x_i, y_i) for training, 10,000 held out for testing.
- **Model:** a family of functions f_θ indexed by a parameter vector θ.
- **Loss:** small when $\hat{p}$ puts high probability on the true digit.
- **Learning:** choose θ to make the average training loss small, then measure on the held-out images.
- **The econometrician's problem with a richer f_θ: a specification, a criterion, an estimator.**

Try Writing the Rule for a “3”



four people, one digit

- A first rule: “two open curves, stacked”. Now define *curve*, *open* and *stacked* in ink levels.
- Slant, stroke width, size and position change all 784 numbers and none changes the digit.
- **Polanyi’s paradox**: we know more than we can tell.
- **When the rule cannot be written, give examples and let the rule be estimated.**

© Zhigang Feng
Redistribution prohibited without permission

Source: M. Polanyi, *The Tacit Dimension*, 1966; D. Autor, “Polanyi’s paradox and the shape of employment growth”, NBER WP 20485, 2014.

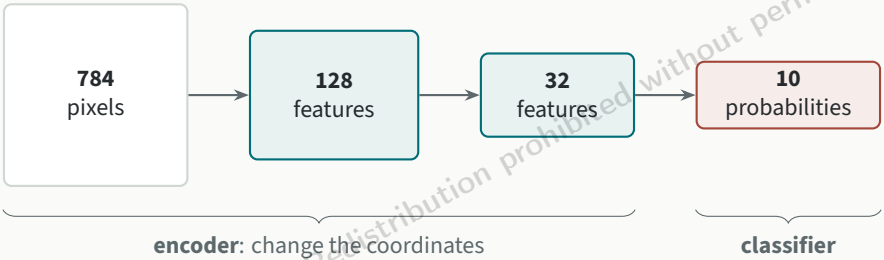
Why It Is Possible at All: Structure

- There are 256^{784} possible 28×28 images. Almost all of them look like television static.
- Images of digits are a vanishing fraction of that space, and they are not scattered at random: strokes are continuous, connected and few.
- The **manifold hypothesis**: real high-dimensional data lie near a surface of much lower dimension inside the space they are recorded in.
- Macroeconomists use the same idea. Krusell and Smith (1998) replace an infinite-dimensional wealth distribution by its mean, because in equilibrium the distribution moves in very few directions.
- **Learning in high dimension works when the data have low-dimensional structure, and fails when they do not.**

© Zhiyang Feng • Redistribution prohibited without permission

Source: P. Krusell and A. Smith, "Income and wealth heterogeneity in the macroeconomy", *JPE* 106(5), 1998.

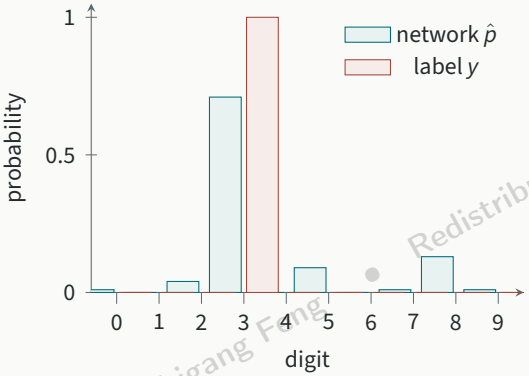
The Encoder View



© Zhirang Feng Redistribution prohibited without permission

- Each layer re-describes the input in fewer, more useful numbers. The last layer is an ordinary multinomial logit on those numbers.
- The encoder and the classifier are estimated *jointly*, by the same criterion.
- In econometric terms: the regressors are estimated, not chosen by the researcher.
- **A deep network is a logit whose explanatory variables are learned.**

Labels, One-Hot Targets, and a Probability for Each Digit



© Zhigang Feng. Redistribution prohibited without permission

- The label "3" is a **one-hot** vector: a 1 in position 3, zeros elsewhere.
- The **softmax** turns ten scores z_j into probabilities, $\hat{p}_j = e^{z_j} / \sum_k e^{z_k}$: McFadden's multinomial logit, with scores as utilities.
- The loss is the negative log-likelihood of the true digit: $-\log 0.71 = 0.34$ here.
- **Classification is maximum likelihood for a discrete-choice model.**

What the Input's Structure Decides

Input	Structure it carries	Architecture	Economic example
image	neighbours matter	convolutional	night lights
sequence	order matters	recurrent, transformer	policy statements
set, network	order is irrelevant	graph network	households
state vector	none assumed	dense	(k, z) in a growth model

- Decks 2.1 to 2.5 use the dense network, which assumes nothing. Deck 2.6 returns to the other rows.
- **An architecture is an assumption about the input, written into the function.**

© Zixiang Feng Redistribution prohibited without permission

Session 2.1

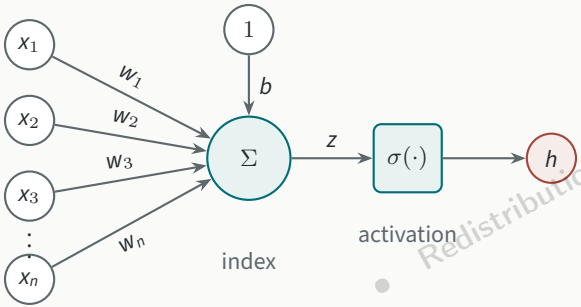
The Neuron

A regression with a nonlinear output



Redistribution prohibited without permission
© Zhigang Feng

A Neuron Is a Regression with a Nonlinear Output

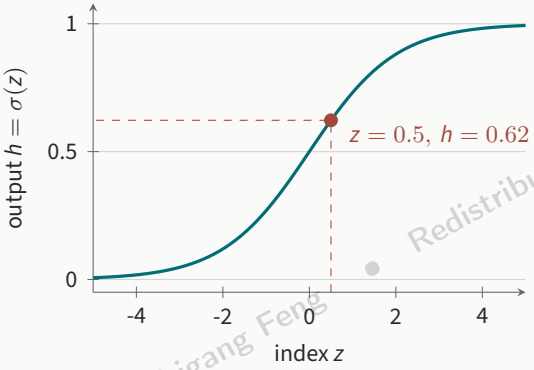


- Step 1, a linear index: $z = w^\top x + b$. The **weights** w are slopes; the **bias** b is the intercept.
- Step 2, a fixed nonlinear function, the **activation**: $h = \sigma(z)$.
- With $\sigma(z) = 1/(1 + e^{-z})$ this is exactly a logit model.
- The parameters are $\theta = (w, b)$: $n + 1$ numbers.
- **One neuron is a generalised linear model. Nothing more.**

© Zhigang Feng
Redistribution prohibited without permission

Source: Miao & Feng, *AI for Economists*, §9.1.

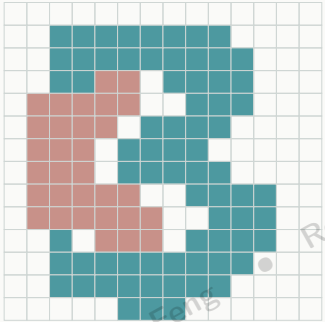
One Neuron, with Numbers



© Zhigang Feng
Redistribution prohibited without permission

- Inputs $x = (1.0,\ 0.5,\ 0.0)$; weights $w = (0.8,\ -0.4,\ 0.3)$; bias $b = -0.1$.
- Index: $z = 0.8(1.0) - 0.4(0.5) + 0.3(0.0) - 0.1 = 0.5$.
- Output: $h = \sigma(0.5) = 1/(1 + e^{-0.5}) = 0.62$.
- Read it as a probability: the neuron is "62% on".
- The third input is zero, so w_3 played no part. **A weight matters only where its input is active.**

What the Weights Are: a Template Laid over the Image

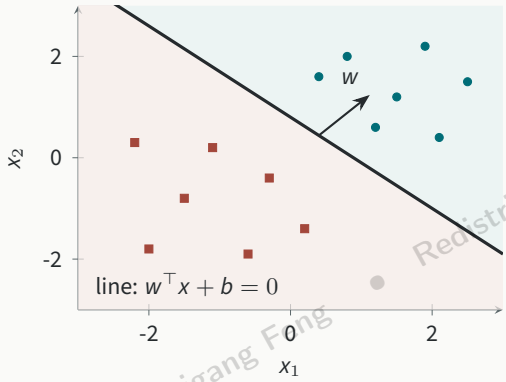


teal: positive weight **red**: negative weight

- A neuron reading the image has one weight per pixel, so its weights can be drawn as an image.
- $w^\top x$ is large when ink falls on positive weights and stays off negative ones.
- This one rewards ink where a “3” has strokes and penalises ink on the left, where an “8” closes its loops.
- **One neuron asks one question of the input: how much do you look like my template?**

© Zheng Feng Redistribution prohibited without permission

One Neuron Draws One Line



© Zhigang Feng
Redistribution prohibited without permission

- $\sigma(z) > 0.5$ exactly when $z = w^\top x + b > 0$.
- The neuron switches on in a half-space whose edge is the **hyperplane** $w^\top x + b = 0$: a line in two dimensions.
- w is perpendicular to the line; b slides it. The activation sets how sharply the output changes across the line, and does not bend it.
- **One neuron can separate only what a single straight cut can separate.**

An Economist Has Met This Before: Logit

- **Binary choice.** $\Pr(y = 1 \mid x) = \sigma(w^\top x + b)$ is the logit model; $w^\top x + b$ is the difference in systematic utility.
- **Estimation.** Maximum likelihood on the logit is minimising **cross-entropy**, the loss used to train classifiers. Same criterion, different name.
- **Vocabulary.** weights = coefficients; bias = intercept; activation = link function; training = estimation; inference = prediction.
- **What is new** is not the unit. It is stacking many units so that the regressors of the last one are themselves estimated.
- **Everything you know about specification, identification and out-of-sample testing still applies.**

© Zhiyang Feng · Redistribution prohibited without permission

Source: D. McFadden, "Conditional logit analysis of qualitative choice behavior", in *Frontiers in Econometrics*, 1974.

Takeaways, and What Deck 2.2 Opens

- **A learning problem** is data, a family f_θ, a loss, and a rule for judging the result.
- **An image is a vector**, and learning works because real data occupy a small, structured part of the space.
- **A neuron is a logit**: a linear index through a fixed nonlinearity, $h = \sigma(w^\top x + b)$.
- **Alone it draws one line**, and one line is not enough for the digits or for XOR.
- **The whole of a network's flexibility sits in that σ.** We have used it without looking at it.
- **Next, Deck 2.2: what σ is, why a layer without it buys nothing, and what a network with it can reach.**

© Zhirang Feng Redistribution prohibited without permission